

Exercícios — Arithmetic Intensity & Roofline Model

10 exercícios de síntese cobrindo cálculo de AI na mão, ridge point, fórmula do roofline e diagnóstico de kernel. Resolva sem olhar a seção de respostas no final.

1. Redução (soma de array) AI na mão — reforço do método

```
float sum_array(float* x, int n) {  
    float sum = 0;  
    for (int i = 0; i < n; i++)  
        sum += x[i];  
    return sum;  
}
```

Assuma float32 (4 bytes). O acumulador sum fica em registrador — não é lido/escrito na memória global a cada iteração, só $x[i]$ é lido.

1. Quantos FLOPs totais (em função de n)?
2. Quantos bytes movidos (em função de n)?
3. Qual a AI dessa operação?

2. Matmul em batch generalização para tensores com mais dimensões

Tensores A e B com shape (Batch, N, N). Operação: $C[b] = A[b] @ B[b]$ para cada b de 0 a Batch-1, gerando C com shape (Batch, N, N). Caso ingênuo (sem reuso de cache), mesma suposição que usamos no matmul simples.

1. Classifique as dimensões: qual é de batch, qual é de contração, quais são de saída?
2. FLOPs totais em função de Batch e N.
3. Bytes totais (caso ingênuo) em função de Batch e N.
4. AI da operação — ela depende de Batch? Por quê?

3. Convolução 2D fechando o cálculo completo, com números reais

```
for (int i = 0; i < N-K+1; i++)  
    for (int j = 0; j < N-K+1; j++) {  
        float sum = 0;  
        for (int ki = 0; ki < K; ki++)  
            for (int kj = 0; kj < K; kj++)  
                sum += img[i+ki][j+kj] * kernel[ki][kj];  
        out[i][j] = sum;  
    }
```

$N = 256$, $K = 5$ (imagem 256×256 , kernel 5×5 , float32). Caso ingênuo: cada pixel de saída relê a janela $K \times K$ de img e o kernel $K \times K$ inteiro, do zero.

1. FLOPs totais (número, não só fórmula).

- Bytes totais movidos (número).
- AI da operação.
- Hardware fictício: pico 20 TFLOP/s (FP32), banda 1,5 TB/s. Calcule o ridge point e classifique o regime dessa convolução.

4. Ridge point de hardwares reais cálculo direto

Hardware	Pico de compute	Banda de memória
X (tensor core, FP16)	60 TFLOP/s	2 TB/s
Y (sem tensor core, FP32)	9,7 TFLOP/s	900 GB/s

- Calcule o ridge point de X.
- Calcule o ridge point de Y.
- Por que os dois ridge points são tão diferentes, mesmo a banda não sendo tão diferente assim entre os dois?

5. Classificando regime de várias operações aplicação direta

Use o ridge point do Hardware Y calculado no exercício 4.

Operação	AI (FLOP/byte)
elementwise add	0,15
matmul bem tileado	45
attention QK^T	8
layer norm	0,5

- Classifique cada operação como compute-bound ou memory-bound nesse hardware.

6. Migração de regime — dobrando compute (não banda)

contraste com o exercício anterior sobre bandwidth

Hardware fictício original: pico 100 GFLOP/s, banda 50 GB/s.

- Se você **dobrar o pico de compute** (200 GFLOP/s) mantendo a banda igual (50 GB/s), o ridge point sobe ou desce? Para qual valor?
- Uma operação com AI = 3,0 FLOP/byte — antes dessa mudança, qual regime ela está? E depois? Ela migra de regime? Se sim, de qual para qual?

7. Teto real de performance fórmula do roofline, não só ridge point

Hardware: pico 40 TFLOP/s, banda 1,2 TB/s. Operação com AI = 8 FLOP/byte.

- Calcule o teto de performance alcançável (em TFLOP/s) usando $\min(\text{Peak_FLOPS}, \text{AI} \times \text{Peak_Bandwidth})$.

2. Esse teto vem do lado de compute ou de memória?

8. Porcentagem do teto cuidado com a unidade – não é o ridge point

Use o teto calculado no exercício 7. Você mede essa operação rodando a 7,2 TFLOP/s no hardware real.

1. Que porcentagem do teto isso representa?
 2. Isso está próximo do teto ou ainda tem bastante espaço pra otimizar?
-

9. Roofline vs. ridge point conceitual

1. Em 2-3 frases: qual a diferença entre "roofline" e "ridge point"? O que cada um representa, em que unidade, e o que cada um permite decidir?
-
-

10. Diagnóstico com profiler ALU ociosa ≠ memory-bound automático

Você profila um kernel no Nsight Compute e ele reporta:

- Compute Throughput: 22% do pico
- Memory (DRAM) Throughput: 24% do pico

A AI teórica desse kernel, calculada na mão, é 0,6 FLOP/byte. O ridge point do hardware é 8 FLOP/byte (então, teoricamente, o kernel deveria ser bem memory-bound).

1. Os dois números medidos (22% e 24%) confirmam que o kernel está memory-bound no sentido de "banda saturada"? Justifique.
 2. O que esse padrão (os dois baixos e parecidos) sugere sobre a causa real da baixa performance?
 3. Que tipo de métrica/problema você investigaria primeiro no profiler?
-

Respostas

Só confira depois de tentar resolver. Os números seguem as mesmas convenções usadas nos exercícios anteriores (2 flops por multiplicação-e-soma, 4 bytes por float32, caso ingênuo sem reuso de cache).

1. Redução

FLOPs $\approx n(n-1)$ somas, aproximando pra n em n grande, mesma convenção usada antes | Bytes = $n \times 4 = 4n$ (só leitura de x , sem escrita de array) | **AI = $n / 4n = 0,25$ FLOP/byte.**

Repare: de novo 0,25 — mesmo padrão do axpy e do matmul ingênuo. Qualquer operação com "1 flop por 4 bytes lidos/escritos" cai nessa faixa quando não há reuso.

2. Matmul em batch

Batch = dimensão de batch (independente) | a dimensão "N" que é somada no produto interno = contração | as duas dimensões de saída (linhas e colunas de C) = saída.

FLOPs = $\text{Batch} \times 2N^3$ | Bytes (ingênuo) = $\text{Batch} \times N^2 \times 2N \times 4 = \text{Batch} \times 8N^3$

AI = $(\text{Batch} \times 2N^3) / (\text{Batch} \times 8N^3) = 2/8 = 0,25$ — o Batch cancela! A AI não depende do tamanho do batch, porque batch multiplica FLOPs e bytes igualmente. Confirma a regra: dimensões de batch não mudam a proporção, só escalam os dois lados juntos.

3. Convolação 2D (N=256, K=5)

$M = N - K + 1 = 252$ | FLOPs = $M^2 \times 2K^2 = 63.504 \times 50 = \mathbf{3.175.200 \text{ FLOPs}}$

Bytes = $M^2 \times (8K^2 + 4) = 63.504 \times 204 = \mathbf{12.954.816 \text{ bytes } (\sim 12,35 \text{ MB})}$

AI = $3.175.200 / 12.954.816 \approx 0,245$ FLOP/byte

Ridge point = $20e12 / 1,5e12 \approx 13,3$ FLOP/byte. Como $0,245 \ll 13,3 \rightarrow$ **memory-bound**, igual ao matmul ingênuo.

4. Ridge points

Ridge X = $60e12 / 2e12 = \mathbf{30 \text{ FLOP/byte}}$

Ridge Y = $9,7e12 / 900e9 \approx \mathbf{10,78 \text{ FLOP/byte}}$

São tão diferentes porque tensor cores aumentam o pico de *compute* muito mais do que aumentam a banda — o numerador da razão cresce muito mais que o denominador, empurrando o ridge point pra direita (precisa de mais reuso de dados pra virar compute-bound).

5. Classificando regimes (ridge Y $\approx 10,78$)

elementwise add (0,15) $\rightarrow$ memory-bound | matmul tileado (45) $\rightarrow$ compute-bound | attention QK^T (8) $\rightarrow$ memory-bound (8 < 10,78, mas perto da fronteira) | layer norm (0,5) $\rightarrow$ memory-bound

6. Migração dobrando compute

Ridge original = $100/50 = 2,0$. Dobrando compute: novo ridge = $200/50 = \mathbf{4,0}$ — **ridge point SOBE** (direção oposta ao exercício de dobrar banda, que fazia o ridge point descer).

Operação com AI=3,0: antes (ridge=2,0), $3,0 > 2,0 \rightarrow$ era **compute-bound**. Depois (ridge=4,0), $3,0 < 4,0 \rightarrow$ passa

a ser **memory-bound**. Ela migra de compute-bound para memory-bound — o espelho exato do que aconteceu no exercício de dobrar banda (lá, operações migravam de memory-bound para compute-bound).

7. Teto real

$AI \times \text{Peak_BW} = 8 \times 1,2 = 9,6 \text{ TFLOP/s} \mid \min(40, 9,6) = \mathbf{9,6 \text{ TFLOP/s}}$.

Vem do lado de **memória** ($9,6 < 40$, o menor dos dois lados é o que limita).

8. Porcentagem do teto

$\% = 7,2 / 9,6 = \mathbf{75\%}$.

Está relativamente próximo do teto (75%), mas ainda há ~25% de espaço até o limite teórico daquela AI específica.

Note que a divisão é **medido (TFLOP/s) / teto (TFLOP/s)** — nunca contra o ridge point (que estaria em FLOP/byte, unidade incompatível).

9. Roofline vs. ridge point

Roofline é a função/modelo teórico completo que, para qualquer AI, prevê o teto de performance em GFLOP/s (ou TFLOP/s): $\min(\text{Peak_FLOPS}, AI \times \text{Peak_BW})$. **Ridge point** é só um valor fixo de AI (em FLOP/byte) — o ponto onde as duas retas do roofline se cruzam — usado apenas para classificar se uma operação é compute-bound ou memory-bound, sem gerar nenhum número de performance por si só.

10. Diagnóstico com profiler

Não — os dois números (22% e 24%) **não confirmam** memory-bound no sentido de "banda saturada". Se fosse realmente memory-bound por saturação de banda, o Memory Throughput deveria estar perto de 100% (saturado), com o Compute Throughput baixo. Aqui os dois estão baixos e parecidos, o que sugere que **nenhum dos dois recursos está sendo usado plenamente** — o problema real provavelmente é **ocupância/latência** (poucos warps ativos pra esconder a latência de memória), divergência de warp, ou stalls de sincronização — não falta de banda em si.

Primeiro passo de investigação: olhar **occupancy** (limitada por registradores/shared memory?) e os **warp stall reasons** no Nsight Compute (ex: "stall long scoreboard" indicaria espera por memória por falta de paralelismo suficiente, não por banda saturada).